

From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Feng Shuo

September 28, 2026

Abstract

Large language models produce mathematical arguments that are fluent but not trustworthy: a proof may read correctly and still be wrong. We argue that this failure is not primarily a failure of generation, nor of verification, but of the *middle layer* that connects them. We decompose an AI4Math system into three layers — generation, semantic reduction, and verification — and show that the second layer is the actual bottleneck: verification can only operate on computable objects, and natural language is not one. We then make the argument operational in two ways. First, we give a taxonomy of four reduction failure classes, each with a machine-checkable detection criterion, showing that all four are invisible to a stronger prover. Second, we introduce *certified accuracy* as a metric distinct from reported accuracy, and prove an identity that decomposes the latter into an auditable part and an unauditable part. The practical consequence is that a system can improve its reported accuracy while becoming *less* verifiable, and that current benchmarks cannot detect this. We corroborate the bottleneck claim in two independent ways. Published end-to-end evidence composes a 97%-accurate autoformaliser with a 69%-accurate prover and obtains roughly 36% end-to-end accuracy, the loss being traced to formal-informal mismatch. And we report a first measurement of our own: across GSM8K and two MATH subjects, one model’s reported accuracy falls by less than five points ($0.980 \rightarrow 0.933$) while the share of its own correct answers that survives machine audit falls from 0.67 to 0.12. The failures concentrate in the reduction layer and none of them is visible to a stronger verifier. We also find that ν is not stable across runs of a fixed hosted model, moving by up to 16 points, which is itself an argument for measuring it rather than assuming it. We do not claim a new solver; we claim a sharper decomposition, a testable taxonomy, and a measurable target.

Keywords: AI for mathematics, formal verification, semantic reduction, neuro-symbolic systems, reliability, evaluation methodology

1 Introduction

Machine learning has entered core mathematical practice. Neural models guided human intuition toward new results in knot theory and representation theory [2]; an evolutionary procedure paired with a program evaluator discovered new cap-set constructions beyond the best known ones [8]; and a neuro-symbolic system solved 25 of 30 olympiad geometry problems [10].

Yet these successes sit next to an uncomfortable observation. Language models regularly emit reasoning chains that are locally plausible and globally wrong [11]. On competition-level benchmarks such as MATH [4] and GSM8K [1], a correct final answer does not certify a correct derivation. Process supervision, which labels intermediate steps rather than only outcomes, improves reliability [5] — but it is expensive precisely because there is no machine-checkable handle on an intermediate step written in natural language.

The field’s dominant response has been to add verification on top of generation. We contend that this response is aimed one layer too high. Verification instruments operate on objects that already possess a decidable notion of well-formedness; a natural-language derivation possesses

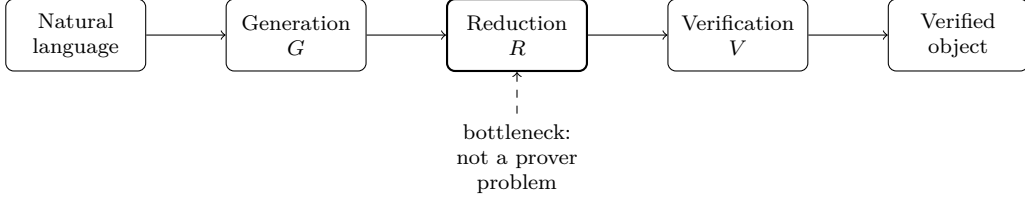


Figure 1: The three-layer decomposition of an AI4Math system. Natural language is the interface; the reduced object is the kernel. Our claim is that failures concentrate in R , and that strengthening V cannot compensate.

none. Something must therefore stand between the two, and that something is where we claim the failures concentrate.

Claim. Reliability cannot be bought by final-answer verification alone. It requires an explicit *semantic reduction layer* that maps natural-language reasoning into a structured object a verifier can act on — and that layer must be measured, not assumed.

Contributions.

1. A three-layer decomposition of AI4Math systems that isolates semantic reduction as a first-class component rather than as preprocessing.
2. A structural argument, plus a decomposition identity (Section 3), showing that most observed failures concentrate in that layer.
3. A taxonomy of four reduction failure classes, each paired with a machine-checkable detection criterion (Section 4).
4. *Certified accuracy* as a metric distinct from reported accuracy, a reliability checklist, and an evaluation protocol that reports both (Sections 5 and 6).

We are explicit about scope: this paper proposes no new solver. It is analytic in its main contribution — a sharper decomposition and metrics that make the central claim falsifiable — but Section 6 does report measurements, on one hosted model and three school-level corpora, and Section 6.6 is explicit about how much of what we measured belongs to our apparatus rather than to the model.

2 A Three-Layer Decomposition

Let an AI4Math system be decomposed as

$$G \longrightarrow R \longrightarrow V, \tag{1}$$

where G is generation, R is semantic reduction, and V is verification. Figure 1 places the three layers in their usual pipeline setting, where natural language serves as the *interface* at both ends and the reduced object is the *kernel* in the middle.

2.1 Layer 1: Generation

A language model emits a natural-language reasoning chain, typically via chain-of-thought prompting [11]. Its output is probabilistic, untyped, and semantically underspecified. Nothing in this layer offers a handle by which a machine could reject a step.

2.2 Layer 2: Semantic Reduction

Natural language is mapped to a structured representation: first- or higher-order logic expressions, executable programs, proof objects, or a typed intermediate representation. The theoretical anchor is the Curry–Howard correspondence, under which a proof is a program and a formula is a type. The output of this layer is the *only* thing Layer 3 will ever see.

2.3 Layer 3: Verification

A formal system checks self-consistency, derivability, factual grounding, or constraint satisfaction. In practice this is an interactive theorem prover such as Lean 4 [3] over the mathlib library [9], or a symbolic/SMT solver. The cross-system benchmark miniF2F [12] shows that this layer is now measurable and comparable across provers.

A useful contrast is physics-informed neural networks [7], where the “verifier” is not a proof kernel but a continuous residual: the network is penalized for violating a differential equation. This shows that V need not be symbolic, but it does not change the requirement that whatever V consumes must already be a formal object. The residual is computable precisely because the PDE was written down formally in advance.

3 The Semantic Reduction Bottleneck

We claim that when an AI4Math system fails, it fails in Layer 2 more often than in Layers 1 or 3.

3.1 The structural argument

Verification is only defined over objects with a decidable notion of well-formedness. Natural language has none. Therefore, if Layer 2 produces no object, or produces an object that does not faithfully encode the intended claim, then Layer 3 has nothing to check — or checks the wrong thing. Two consequences follow.

1. **Verification failure masquerading as reasoning failure.** The pipeline reports “unverified” rather than “unreduced”, conflating a translation failure with a mathematical one. The diagnostic signal is destroyed at the point where it would be cheapest to act on.
2. **Undetectable hallucination.** An unfaithful translation can be internally consistent and still prove a proposition different from the one that was asked. A sound kernel certifies the object it was given, not the intention behind it.

This reframes the reliability problem: the scarce resource is not the prover’s power but the *fidelity of the translation*.

3.2 A decomposition identity

The argument becomes testable once written as an identity. Fix a benchmark of N problems. Let $V \subseteq \{1, \dots, N\}$ be the subset for which reduction yields a well-formed, machine-checkable object, and let $\nu = |V|/N$ be the *verifiability rate*. Let a be reported final-answer accuracy, let

$$\alpha = \frac{1}{N} |\{i \in V : \text{object verified and consistent with intent}\}| \quad (2)$$

be *certified accuracy*, and let u be the accuracy on the complement V^c — correct final answers that carry no machine-checkable warrant. Then

$$a = \alpha + (1 - \nu)u, \quad \text{and} \quad \alpha \leq a. \quad (3)$$

Equation (3) is an accounting identity, not a model, and its content is entirely in the second term. The quantity $(1 - \nu)u$ is exactly the part of reported accuracy that cannot be audited. A system may therefore *increase* a while *decreasing* ν : it can become better at producing answers and worse at producing reasons that can be checked, and a benchmark reporting only a will record this as progress. No benchmark we are aware of reports ν , so no current leaderboard can detect this regression. This is the concrete sense in which the field is measuring the wrong quantity.

3.3 Empirical support

The argument above is structural. It is corroborated by a recent end-to-end evaluation that measures exactly the quantity we claim is unmeasured. Ospanov, Farnia, and Yousefzadeh [6] place a model in an “olympiad setting”: read the informal statement, formalise it in Lean, prove it, and receive credit only if the proved statement corresponds to the original informal one. Under this protocol, the best pipeline accuracy is about 36%, against component-level accuracies of 97% reported for autoformalisation and 69% for theorem proving. The composition of two strong components yields far less than either. Analysing the failure modes, the authors trace a considerable portion of the drop to discrepancies between formal and informal statements affecting more than half of miniF2F’s problems, and report that only 59% of the formal statements in the test split are correct.

Two readings are possible, and both support our thesis. Under the first, the loss occurs because the formal statements were written carelessly — a benchmark-quality failure. Under the second, it occurs because the autoformaliser and the prover optimise different objectives — a Layer 2 fidelity failure. These are not competing explanations; a benchmark that does not police Layer 2 cannot detect either. Note also that the authors’ corrected benchmark raises end-to-end accuracy substantially, which shows that the loss was never in the prover. We therefore treat this result as evidence that Equation (3)’s unauditable term is not a small correction but potentially the dominant one.

4 Analysis: A Taxonomy of Reduction Failures

The bottleneck claim is only useful if reduction failures can be told apart. We propose four classes, distinguished by *where in the pipeline they become detectable*. The taxonomy is deliberately syntactic: each class is paired with a criterion a system can evaluate without human judgement, so that the classes are reportable rather than merely descriptive.

Table 1 states the four classes with their criteria.

Why the taxonomy matters. Classes A, T, and R are all *pre-verification* failures: by the time an object reaches V , the error has already been compiled into it, and a sound kernel will faithfully certify the wrong claim. Class N is the inverse failure and is the most damaging, because it is invisible in both directions: the proof is valid and the rendering reads correctly, yet the two no longer correspond. None of the four is mitigated by a stronger prover, a larger model, or more search — which is precisely the sense in which Layer 2 is a bottleneck rather than a weak link that better neighbours can compensate for.

Relation to existing error analyses. Prior work on process supervision [5] labels erroneous steps but does not classify *why* a step cannot be checked; our taxonomy is orthogonal to, and composable with, such labels. A step can be judged correct by a human labeler and still fall into class T, because the omission is invisible in natural language.

Table 1: Four classes of semantic reduction failure. The rightmost column is the point of the framework: no class is recoverable by strengthening the verifier.

Class	Failure	Detection criterion	Visible to V ?
A	<i>Ambiguity</i> : one surface form admits several well-typed formal readings	Parse multiplicity > 1 under the target type system	No — V receives one parse
T	<i>Type omission</i> : implicit domain, coercion, or side condition silently dropped	Elaboration leaves unresolved metavariables or inserts defaults	No — V sees a well-typed term
R	<i>Referential drift</i> : anaphora or scope binds to a wrong antecedent across steps	IR graph retains unresolved or multi-candidate coreference edges	No — V checks the resulting graph
N	<i>Non-invertibility</i> : the verified proposition differs from the intended one	Round-trip mismatch between $\text{render}(\text{parse}(x))$ and x	No — V proves what it was given

5 Proposed Framework

5.1 Typed intermediate representation

We propose a typed IR with four components: a type system, an operational semantics, a constraint system, and a composable graph/AST structure. The IR is not a new logic; it is the interface contract that makes Layers 1 and 3 separately auditable. We sketch it as a small explicitly typed term language with types τ , terms e , and propositions ϕ :

$$\begin{aligned}
\tau &::= \mathbb{N} \mid \mathbb{Z} \mid \mathbb{Q} \mid \mathbb{R} \mid \tau \rightarrow \tau \mid \text{Set}(\tau) \\
e &::= x \mid n \mid e + e \mid e \times e \mid e/e \mid \text{apply}(e, e) \mid \text{let } x : \tau = e \text{ in } e \\
\phi &::= e \leq e \mid e = e \mid \phi \wedge \phi \mid \phi \Rightarrow \phi \mid \forall x:\tau. \phi
\end{aligned}$$

with the side condition that every division carries an explicit non-zero proof obligation. That obligation is exactly the one whose silent discharge constitutes class T of Section 4. A term counts as well-formed only if elaboration discharges every such obligation; a reduction that leaves any obligation unresolved is reported as *unreduced*, never as verified.

5.2 Bidirectional consistency

We require $\text{IR} \leftrightarrow \text{natural language}$ to be both interpretable and traceable: every IR node records the span it came from, and every rendered sentence records the node it came from. This property is what makes human audit possible, and it is the mechanism that exposes class N failures through round-tripping.

5.3 A reliability checklist

The framework is only as good as its enforcement. We state five requirements as a checklist, each traceable to a failure class above.

1. **Provenance.** Every IR node carries the source span it was reduced from. (Enables audit; exposes class R.)
2. **Explicit elaboration.** Implicit domains, coercions, and side conditions are elaborated or flagged. Silent defaults are prohibited. (Exposes class T.)

Table 2: Reported metrics. ν , κ , and α are defined only over outputs that reach Layer 3; a is defined over all outputs.

Symbol	Name	Definition
a	Accuracy	Fraction of final answers matching the reference
ν	Verifiability rate	Fraction of outputs for which R yields a well-formed object
κ	Consistency	Fraction of verified objects whose rendering matches the intended claim
α	Certified accuracy	Fraction of <i>all</i> outputs that are verifiable, verified, consistent, and correct

3. **Parse determinism.** A reduction with multiplicity > 1 is reported as ambiguous, not resolved by sampling. (Exposes class A.)
4. **Round-trip audit.** $\text{render}(\text{parse}(x))$ is compared against x ; mismatches are reported, never repaired silently. (Exposes class N.)
5. **Reject, do not repair.** A failed reduction reports “unreduced”. It must never be reported as “verified” or silently downgraded to an unchecked answer.

Requirement 5 is the one most likely to be violated in practice, because it lowers reported accuracy. We regard that as the point: the checklist trades apparent performance for auditable performance.

6 Evaluation Protocol

6.1 Benchmarks

GSM8K [1] and MATH [4] for informal reasoning; miniF2F [12] for formal proving. We note a methodological caveat discussed in Section 7.2: miniF2F statements are human-formalized, so it exercises Layer 3 but not Layer 2.

Two filters are applied to every task and must be stated because they shape the numbers. First, only problems whose reference answer is an integer are scored. This is a grading decision, not a modelling one: a reference of $1/11$ is stored as 0.09090909090909091, so a model that correctly writes 0.0909 is off by 9.1×10^{-6} and would be marked wrong for writing fewer digits. Restricting to integers makes “correct” a fact about the answer rather than about its decimal rendering. It removes 14.1% of MATH prealgebra and 8.3% of number theory. Second, we take the first 200 surviving problems of each test split rather than sampling, so the subset is reproducible without a seed.

6.2 Metrics

Table 2 defines the reported quantities. The protocol’s only hard requirement is that a is never reported alone.

6.3 Comparison

A pure LLM baseline against an IR-based pipeline, with ν and α reported for both. The prediction implied by Section 3 is falsifiable: the IR pipeline should show lower a but strictly

Table 3: Measured results on three task distributions, one model (`deepseek-v4-flash`, temperature 0), one harness, one session. The baseline column is free-form chain-of-thought; the IR columns are the same model asked to emit the typed IR of Section 5. κ is conditioned on outputs that parse and evaluate; α is over all outputs. $n < 200$ for the MATH rows because problems with a non-integer reference answer are excluded (Section 6.1).

Task	n	baseline a	IR a	ν	κ	α
GSM8K	200	0.980	0.680	0.835	0.825	0.660
MATH, prealgebra	170	0.959	0.500	0.618	0.848	0.435
MATH, number theory	195	0.933	0.349	0.626	0.592	0.113

positive ν and α , whereas the baseline should show $\nu = \alpha = 0$ regardless of a .

6.4 An illustrative calculation

The identity can also be exercised on figures published elsewhere. We label this an illustration, not one of our measurements. Using the component accuracies reported for the miniF2F pipeline [6] — 97% for autoformalisation and 69% for theorem proving — a naive composition under an independence assumption gives $0.97 \times 0.69 \approx 0.67$. The same work reports end-to-end accuracy of about 0.36 under the stricter criterion that credit requires the proved statement to correspond to the original informal one. Read through Equation (3), the unauditable term is therefore of order $0.67 - 0.36 \approx 0.31$: roughly thirty points of nominally reported accuracy carry no machine-checkable warrant.

Two caveats are binding. The independence assumption is ours and is not defended by that work; and the correspondence criterion behind 0.36 is stricter than the one implicit in 0.67, so the two quantities are not strictly like-for-like. We offer the calculation as an order-of-magnitude indication.

6.5 Results

Table 3 fills in the reporting form with measured values. Its hard requirement is unchanged: no cell may be inferred from another, so α is counted rather than derived from a and ν .

Reading the table. The baseline column is nearly flat: 0.980, 0.959, 0.933. Read on its own it says that the model is a competent solver on all three distributions and only slightly worse on the harder ones. The α column describes the same model on the same problems and says something else: 0.660, 0.435, 0.113. Reported accuracy falls by 4.7 points across the three rows; certified accuracy falls by 55. Nothing in the first column would let a reader anticipate the third.

Paired analysis. A low ν admits two explanations that ν alone cannot separate: the problem is genuinely hard, or the model can solve it but cannot express the solution in a checkable form. Table 4 removes the first explanation by restricting every quantity to the subset of problems the *baseline* answered correctly. On that subset the model demonstrably knows the answer, so whatever the IR condition fails to produce is not a failure to find one.

This is the paper’s central measurement. Of the answers the model can produce, the share that survives machine audit falls from two thirds on grade-school arithmetic to one eighth on number theory, while the share it gets right in the first place falls by less than five points. The two quantities describe the same model on the same problems and they move in opposite directions.

Table 4: Paired results, restricted to the problems the baseline solved (“s”). Task difficulty is controlled by construction: every problem in this subset was answered correctly by free-form decoding.

Task	solved	$\nu \mid s$	$a_{\text{IR}} \mid s$	$\alpha \mid s$
GSM8K	196	0.837	0.689	0.668
MATH, prealgebra	163	0.632	0.521	0.454
MATH, number theory	182	0.626	0.363	0.121

Table 5: Failure classes. “contract” counts class T rejections for omitting the non-zero proof required on division; “out of grammar” counts class R rejections naming an operation the IR cannot express (gcd, mod, $\lfloor \cdot \rfloor$, $\sqrt{\cdot}$). Both are partly artefacts of our IR rather than semantic failures, and Section 6.6 prices them.

Task	parse-fail	contract (T)	grammar (R)	post-parse (N)
GSM8K	33/200	26	3	30
MATH, prealgebra	65/170	24	27	21
MATH, number theory	73/195	19	26	77

Where the failures land. Table 5 splits the failures into the classes of Section 4, separating two things that are easy to conflate: outputs that never became a checkable object, which is what lowers ν ; and outputs that parsed but then failed to evaluate or whose goal disagreed with their own stated answer, which is class N and does not lower ν .

The informative entry is the last column. These are outputs the checker accepted as well-formed and then evaluated to a value different from the number the model itself reported — the model asserts one thing and its own structured derivation asserts another. On GSM8K and prealgebra this happens to about one output in seven; on number theory, to two in five. The failures concentrate exactly where Section 4 predicts, in the mapping rather than in search: no output in any condition failed because a proof could not be found.

6.6 How much of the failure is our IR’s fault?

Two entries of Table 5 are artefacts rather than findings, and we price them instead of reporting them as reduction failures.

The “contract” column counts outputs rejected because a division lacked the non-zero proof the schema demands. The rule is ours; a model that ignores it has failed to follow a contract, not necessarily to preserve meaning. Waiving it raises ν from 0.835 to 0.965 on GSM8K, from 0.618 to 0.759 on prealgebra, and from 0.626 to 0.723 on number theory. The gap between GSM8K and MATH survives the waiver almost unchanged, so the finding does not depend on that rule.

The “out of grammar” column is a stricter objection. Our IR expresses arithmetic over rationals and nothing else, so a problem needing gcd, modular arithmetic, or a floor cannot be expressed in it at all; rejecting such an output measures our grammar, not the model. These are 1.5% of GSM8K, 15.9% of prealgebra and 13.3% of number theory. We report this rather than filtering it away, because a reduction layer that cannot carry the semantics of its domain is a real limitation of the layer — but the honest reading is that part of the MATH deficit belongs to the IR design and not to the model.

What remains after both are set aside is the post-parse column, and it is the one that grows.

6.7 Reproducibility

ν turned out to be less stable than we expected, and the instability is a result rather than a nuisance. Across repeated runs of the identical configuration on GSM8K — same 200 problems, same model string, same temperature, same harness — we measured $\nu = 0.995, 0.835, 0.800, 0.810$ and 0.840 . The first of these was taken about forty minutes before the others; the four later runs agree with each other to within 0.04 and disagree with the first by 0.16. We did not change the prompt, the corpus, or the checker in between, and a control run with the original phrasing restored gave 0.840, so the drift is not attributable to our prompt edit. Repeated runs on the MATH subsets are tighter: $\nu = 0.641, 0.618, 0.647$ on prealgebra and $0.579, 0.626, 0.600$ on number theory.

Two consequences follow. First, a single ν measured against a hosted model should not be read as a property of that model; the ranges above are disjoint between GSM8K (0.80–0.84, excluding the drifted first run) and MATH (0.58–0.65), which is why we claim a task effect at all, but any individual number carries an uncertainty of a few points that we cannot attribute to a cause. Second, and more to the point of this paper: a quantity that moves 16 points under a fixed model version is exactly the kind of quantity that must be reported rather than assumed, and it is not reported today.

Falsification criteria. We state in advance what would refute the bottleneck hypothesis, so that the claim is testable rather than merely plausible:

- If $\nu \approx 1$ and $\alpha \approx a$ for a system that performs no explicit reduction, then reduction is not a bottleneck and the decomposition adds nothing.
- If observed failures do not concentrate in classes A–N but appear instead as ordinary proof-search failures, the taxonomy is not carving at the joint.
- If enforcing the checklist raises reported accuracy a , then our claim that auditable performance trades against apparent performance is wrong.

Execution status. The numbers in Tables 3–5 are measured, not illustrative. One hosted model (`deepseek-v4-flash`) at temperature 0 produced every completion behind them — 1,130 for the three tasks of Table 3 (565 problems $\times$ 2 conditions) and 4,090 in total across the eleven runs of Section 6.7 — with no call failures. The harness, the corpora, and the verbatim model outputs are distributed with this paper, so every ν and κ reported here can be re-derived without trusting us. The calculation of Section 6 remains an illustration performed on figures published by other work and is not one of our measurements.

Three observations. *Reported accuracy hides a collapse, and the two move in opposite directions.* Across the three tasks the model’s free-form accuracy falls $0.980 \rightarrow 0.959 \rightarrow 0.933$, a change of under five points that any reasonable reader would describe as mild degradation. Over the same tasks the share of its own correct answers that survives audit falls $0.668 \rightarrow 0.454 \rightarrow 0.121$. The first column says the model is fine; the second says most of what it knows cannot be warranted, and that the problem gets worse precisely as the semantics get harder. Only the second is visible in α .

The cost of auditability is real, and we no longer claim it is free. An earlier single run of this protocol on GSM8K gave $a = 0.975$ for the IR condition against 0.985 for the baseline, and we reported the one-point difference as evidence that a checkable pipeline need not sacrifice apparent accuracy. That result did not replicate: across four subsequent runs the IR condition scores 0.680, 0.665, 0.680 and 0.675 against baselines of 0.980, 0.975, 0.975 and 0.975. The IR pipeline is *substantially* worse on reported accuracy on every task — by 30 points on GSM8K, 46 on prealgebra and 58 on number theory — and McNemar exact tests give $p < 10^{-16}$ on all

three, so this is not a sampling artefact. We withdraw the earlier claim. Note that this does not trigger falsification criterion (iii), which predicted the opposite sign; what it changes is the practical reading. Enforcing semantic reduction is expensive in the only currency benchmarks currently track.

The taxonomy carves at a joint, and the joint is not where accuracy lives. Every failure we observed fell in classes A, T, R or N. None was a proof-search failure, none was an arithmetic error the checker could have caught by recomputing, and none would have been detected by a stronger verifier, because in every case either no object reached the verifier or the object that reached it was internally consistent with itself and merely failed to say what the model meant. The concentration in class N on number theory — 77 outputs that parsed, evaluated, and then disagreed with the model’s own answer — is the clearest instance: the model asserted one number and derived another, and only the reduction-layer check could see it.

What this run does not establish. Four limits are binding. First, the baseline’s $\nu = 0$ is *definitional, not measured*: the baseline protocol requests free-form output, so no structured object exists to check. We did not discover that the baseline cannot produce one; we specified that it does not. All empirical content of the ν column lies in the IR row. Second, our IR expresses arithmetic over the rationals and nothing else, so part of the MATH deficit measures our grammar rather than the model; Section 6.6 bounds this at 13–16% of MATH problems and shows the task gap survives waiving the other artefact, but a richer IR would presumably raise ν and we cannot say by how much. Third, κ compares the IR’s rendering against the model’s *own* stated answer. Where the two agree, as on GSM8K, the check is weak — a consistency test that cannot fail is not yet measuring semantic fidelity. Where they disagree, as on 39% of number-theory outputs, it is informative, but it then measures internal coherence of one generation rather than correspondence to an independently authored formalisation, which we do not have. Fourth, everything here is one model, one IR, and one family of school-level tasks; nothing licenses a claim about formalising research mathematics into Lean, where the reduction problem is harder and the objects are richer.

7 Related Work

We organise prior work by which layer it strengthens, and for each line ask the same question: *what does it assume is already given?* The answer is uniformly “a formal statement or an executable evaluator” — which is exactly Layer 2’s output. That uniformity is our central finding about the literature.

7.1 Generation-side

Chain-of-thought prompting [11] elicits intermediate steps and improves benchmark accuracy substantially. But what it improves is *plausibility*, not *soundness*: the steps remain natural language and therefore remain outside the reach of any checker. Process supervision [5] is the natural next move — label each step, train a reward model on the labels, and search against it. It works, and PRM800K is a valuable resource. Its two limitations are structural, not incidental. First, cost: labels are produced by human judgement at step granularity, so annotation effort scales with chain length. Second, and more seriously, the label is itself an unverified judgement about a natural-language step. A process reward model learns to predict *what a human annotator would accept*, which is a different target from *what a kernel would accept*. The gap between those two targets is our Layer 2.

7.2 Verification-side

The Lean 4 theorem prover [3] and the mathlib library [9] provide a genuinely sound kernel: a proof that elaborates is correct by construction. What they do not provide is the statement. mathlib is human-written, and formalisation remains the dominant cost in mechanised mathematics — a cost this line of work treats as given.

AlphaGeometry [10] is the most instructive case. It solves 25 of 30 olympiad geometry problems, a striking result. But the geometry problems are translated into a purpose-built domain-specific language by hand-authored translation rules before solving begins. Layer 2 is not solved there; it is *designed away* for one narrow domain. The result therefore does not license any conclusion about reduction quality in general, and generalising it requires solving the very problem it sidesteps.

miniF2F [12] has the same shape. Its formal statements are human-written for each of its 488 problems (244 test, 244 validation), so a system evaluated on miniF2F is evaluated on Layer 3 with Layer 2 supplied by a human. This makes miniF2F an excellent prover benchmark and, for our purposes, a poor reliability benchmark: it cannot observe reduction failure at all.

The point is sharpened by a recent re-examination of the benchmark [6], which finds that only 59% of the formal statements in the test split are correct, with more than half of all problems exhibiting formal–informal discrepancy; the authors release a corrected miniF2F-v2 in response. The lesson generalises beyond one dataset: a benchmark that supplies Layer 2 by hand not only fails to measure it, it fails to notice when it is wrong. We regard this as the most important gap in current evaluation practice.

A different notion of verification appears in physics-informed neural networks [7], where a differential equation residual acts as a continuous, differentiable checker. This widens what V can be — verification without formal proof — but it is subject to the same precondition: the governing equation must be stated in a formal language before a residual can be computed.

7.3 Search-side

FunSearch [8] pairs an LLM with an evolutionary search over programs, using an executable evaluator as the filter, and discovered cap-set constructions surpassing previously known bounds. The evaluator is doing the work of V , and it is doing it soundly. The catch is that this strategy is available only where the problem class admits an executable evaluator: cap sets can be checked by running code. For most of mathematics there is no such evaluator, and there the method has nothing to say. The same observation applies to ML-guided intuition in knot theory [2], which is a tool for generating *conjectures* whose verification remains entirely human.

7.4 Consensus, gap, and positioning

The consensus across all three lines is that verification is valuable and that it should be coupled to generation. The gap is that every instantiation assumes the formal object is already available, and consequently no line of work measures the production of that object. Our positioning follows directly: we do not compete with any prover, search procedure, or supervision scheme. We supply the layer they all presuppose, give it a failure taxonomy, and propose metrics that make its quality visible.

The closest work to ours is the recent re-examination of miniF2F [6], which documents the end-to-end accuracy gap and repairs the benchmark. Our relationship to it is complementary rather than competitive: that work improves a *benchmark*, while we propose a *decomposition and a metric*. After their correction, the benchmark measures Layer 3 more honestly; it still does not report ν or α , so it still cannot distinguish answering well from reasoning well. The other adjacent concern is the formalisation-cost literature around mathlib, which recognises the

problem in practice. Our contribution is to give that concern a place in the system decomposition, a failure taxonomy, and a number attached to it.

8 Discussion

Limitations. IR design is expensive and domain-specific; the cost of faithful semantic mapping may exceed the cost of the proof it enables, in which case the framework is not worth deploying. The taxonomy is syntactic and may not capture semantic errors that preserve well-typedness and round-tripping — a faithful translation of a false conjecture, for instance, is not a reduction failure at all. The framework is silent on conjectures that cannot be stated in the target logic. Finally, Equation (3) bounds but does not estimate the unauditable term; u is not observable without human audit, which is the very thing it is meant to replace.

Threats to validity. We have measured ν and α for one model, one IR design, and three school-level corpora, and the generality of all three choices is untested. The bottleneck hypothesis is supported but not established. What the numbers show is that reduction failures are frequent, that they concentrate in the classes the taxonomy predicts, and that they grow as the semantics get harder; what they do not show is that this pattern survives a different IR, a different model family, or mathematics beyond school level.

Three specific threats deserve to be singled out. The first is *instrumentation*: our IR is arithmetic over the rationals, so a problem needing gcd or modular arithmetic is unrepresentable in it, and a rejection on such a problem measures our grammar. We bound this at 13–16% of MATH (Section 6.6) rather than removing it, because a reduction layer too poor for its domain is a real limitation — but the reader should discount part of the MATH deficit accordingly. The second is *contract compliance*: 26, 24 and 19 outputs were rejected only for omitting a non-zero proof, a rule the model was told to follow and did not. Waiving it narrows the gap between tasks but does not close it. The third is *measurement stability*: ν moved by 16 points between two runs of an identical configuration (Section 6.7). Our task effect is larger than that and survives in repeated runs, but any single ν reported here carries an uncertainty we cannot attribute to a cause, and a reader should not treat the second decimal place as meaningful.

The unauditable term u of Equation (3) is in principle observable once $\nu < 1$; here it is well populated — 33, 65 and 73 outputs per task fell into that stratum — so u can now be estimated rather than assumed, which we leave to follow-up work.

Extensions. The same decomposition should apply to general reasoning systems and to agent architectures, where the “verifier” may be an external tool rather than a proof kernel, and where the corresponding question — did the agent call the tool on the problem it was asked, or on a paraphrase of it? — is a reduction question.

Appendix A turns the specific expressiveness limitation measured in Section 6.6 into a concrete design proposal rather than leaving it as an acknowledged shortcoming.

A Extending the IR to Number-Theoretic Primitives

Section 6.6 reports that 15.9% of MATH prealgebra and 13.3% of MATH number theory problems cannot be expressed in our IR at all: they require greatest common divisor, modular reduction, or the floor function, none of which the grammar of Section 5.1 admits. Those problems were counted as reduction failures, which is correct — but the honest reading is that they measure our grammar, not the model. This appendix sketches the extension that would let the measurement separate the two. We present it as a draft precisely because it is unimplemented: nothing here has been measured, and we do not claim the numbers below as results.

Syntax. Three constructors are added to the term grammar:

$$e ::= \dots \mid \text{gcd}(e, e) \mid e \bmod e \mid \lfloor e \rfloor$$

Typing. The existing judgments are extended as follows:

$$\frac{\Gamma \vdash e_1 : \mathbb{Z} \quad \Gamma \vdash e_2 : \mathbb{Z}}{\Gamma \vdash \text{gcd}(e_1, e_2) : \mathbb{N}} \quad \frac{\Gamma \vdash e_1 : \mathbb{Z} \quad \Gamma \vdash e_2 : \mathbb{Z}}{\Gamma \vdash e_1 \bmod e_2 : \mathbb{N}} \quad \frac{\Gamma \vdash e : \mathbb{Q}}{\Gamma \vdash \lfloor e \rfloor : \mathbb{Z}}$$

The third rule is the interesting one, because it is a *type-lowering* construct: it moves a value from $\mathbb{Q}$ into $\mathbb{Z}$ and therefore discards information. Every existing constructor is type-preserving or type-raising, so adding $\lfloor \cdot \rfloor$ is the first place where the IR admits a term whose denotation is lossy by construction. We flag this because it interacts directly with class N (irreversibility): a proof obligation that the lowered value is the one actually wanted cannot be discharged by typing alone.

Proof obligations. Following the discipline already applied to division, each new constructor carries obligations rather than silent defaults:

- $e_1 \bmod e_2$ requires a witness that $e_2 \neq 0$, of exactly the kind already required for division, and discharged by the same elaboration path. This is deliberately not a new mechanism.
- $\text{gcd}(e_1, e_2)$ requires a witness that e_1 and e_2 are not both zero. Note this is weaker than either argument being non-zero, and getting it wrong is precisely the kind of silent default that Section 5.3 prohibits.
- $\lfloor e \rfloor$ requires no non-zero witness but does require the rounding convention to be stated explicitly, since floor and truncation coincide on the non-negatives and diverge elsewhere.

Cost. The extension is not free, and the cost is not merely implementational. Adding $\bmod$ and gcd keeps the theory inside Presburger arithmetic, which remains decidable, but moves the satisfiability problem into a considerably more expensive fragment; adding $\lfloor \cdot \rfloor$ alongside unrestricted multiplication approaches fragments whose decidability is not something we would want a verification pipeline to assume silently. A conservative deployment is therefore to admit the three constructors into the *syntax* while keeping the verification target restricted to the Presburger fragment, reporting any term outside it as *unreduced* rather than *verified* — the same discipline already applied to unresolved non-zero obligations.

What this would settle. Under the extended grammar the 13–16% currently counted as reduction failures split into two groups with very different meanings: those the model still cannot express (a genuine Layer 2 failure, attributable to the model) and those it can now express and may still get wrong (attributable to the model for a different reason, or to the model’s arithmetic). Until the extension is implemented, that split is not measurable, which is why Section 6.6 reports the whole class as a limitation of the instrument rather than a finding about the model.

B Conclusion

Reliable AI reasoning requires generation, semantic reduction, and verification together. The contribution of this paper is to name the middle term as a first-class component, to give its failures a taxonomy with machine-checkable criteria, and to propose certified accuracy as a quantity that makes the bottleneck hypothesis testable rather than rhetorical. The practical

claim is narrow and we state it plainly: benchmarks that report accuracy alone cannot distinguish a system that reasons well from one that merely answers well, and the distinction is worth measuring. A first measurement makes the cost of ignoring it concrete: as we moved from grade-school arithmetic to number theory, reported accuracy fell by under five points while the proportion of the model’s own correct answers that could be warranted fell from two thirds to one eighth. If that gap is typical, then the accuracy numbers the field currently reports describe a quantity that is largely unwarranted — and, being unwarranted, largely unchecked.

References

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [2] Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, Marc Lackenby, Geordie Williamson, Demis Hassabis, and Pushmeet Kohli. Advancing mathematics by guiding human intuition with ai. *Nature*, 600(7887):70–74, 2021.
- [3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [4] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [5] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [6] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. miniF2F-Lean revisited: Re-viewing limitations and charting a path forward. *arXiv preprint arXiv:2511.03108*, 2025.
- [7] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [8] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [9] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381. ACM, 2020.
- [10] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- [11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [12] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.